Shopify Liquid Mastery: Customize Any Theme Like a Pro

GPLMart - practical digital resources for ecommerce sellers, developers and creators.

v1.0.0 - GPL-3.0-or-later - gplmart.com

Shopify Liquid Mastery

The complete field guide to customizing any Shopify theme — from understanding how Liquid actually works to shipping a custom product page that looks like it cost $5,000. Written for store owners, freelance developers, and agencies who want to stop fighting their theme and start owning it.

Chapter 1: How Liquid Works and Where It Runs

[object Object],[object Object],[object Object]

[object Object]

- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]

The render pipeline

```
Customer request !’ Shopify loads template (JSON or .liquid)
                 !’ Resolves sections + blocks
                 !’ Each section renders its Liquid, pulling in snippets
                 !’ Layout (layout/theme.liquid) wraps everything
                 !’ HTML + CSS/JS assets sent to browser


Understanding this pipeline is the foundation. When something renders wrong, the
question is always: *which layer produced this output?*

> **KEY INSIGHT:** Liquid is not "HTML with variables." It's a render-time engine that
builds HTML. Think of it like a mail-merge for web pages: templates + data objects +
filters = final markup. Anything you can do in the admin theme editor, Liquid can do —
and more.

### Where Liquid runs

| Location | What runs there | Can it use Liquid? |
|---|---|---|
| `layout/theme.liquid` | Global wrapper: `<head>`, header, footer, analytics | Yes |
| `templates/*.liquid` | Classic templates (product, collection, etc.) | Yes |
| `templates/*.json` | OS 2.0 templates — JSON config of sections | Sections render
Liquid |
| `sections/*.liquid` | Reusable page modules with settings_schema | Yes |
| `snippets/*.liquid` | Small reusable fragments, included via `{% render %}` | Yes |
| `assets/*` | CSS, JS, images | No Liquid — plain static files |
| `config/settings_data.json` | Theme settings values | Not Liquid — JSON data |
```

Liquid vs HTML vs CSS vs JavaScript

A common beginner mistake is mixing these up. The rule:

- **Liquid** decides *what content* exists and *which* content.
- **HTML** structures that content.
- **CSS** (in `assets/*.css`) styles it.
- **JavaScript** (in `assets/*.js`) makes it interactive.

If you're writing `if` statements to change pixel colors, you're using the wrong tool. If you're writing CSS to show/hide based on product type, you're using the wrong tool — use Liquid's `if product.type == '...'` instead.

> **PRO TIP:** Write Liquid to output semantic HTML with data attributes (`data-product-id="{{ product.id }}"`), then let CSS and JS do their jobs. This separation is what makes themes maintainable.

Takeaways

- Liquid renders server-side, once per request, into HTML/CSS/JS.
- You can only touch Shopify's exposed object model.
- Keep concerns separated: Liquid for content, CSS for style, JS for behavior.
- `assets/` files are plain static files — no Liquid inside them.

Chapter 2: Theme Anatomy — The Folder Map

Every Shopify theme (including Dawn and all OS 2.0 themes) follows the same folder structure. Learn this map once and every theme becomes readable.

```
theme/
% % %  layout/
%    % % %  theme.liquid          # Global wrapper for every page
% % %  templates/
%    % % %  index.json            # Homepage (OS 2.0)
%    % % %  product.json          # Product page template
%    % % %  collection.json       # Collection page
%    % % %  cart.json             # Cart page
%    % % %  page.json             # Generic pages
%    % % %  blog.json / article.json
%    % % %  search.json
%    % % %  *.liquid              # Classic templates (old themes)
% % %  sections/
%    % % %  main-product.liquid
%    % % %  main-collection.liquid
%    % % %  header.liquid / footer.liquid
%    % % %  *.liquid              # Every reusable module
% % %  snippets/
%    % % %  card-product.liquid
%    % % %  price.liquid
%    % % %  *.liquid              # Small fragments
% % %  assets/
%    % % %  base.css / theme.css  # Compiled styles
%    % % %  theme.js              # Global JS
%    % % %  *.png / *.svg / *.woff2
% % %  config/
%    % % %  settings_schema.json  # Theme settings structure
%    % % %  settings_data.json    # Current setting values
% % %  locales/
%    % % %  en.default.json       # Translation strings
% % %  blocks/                   # Reusable section blocks (OS 2.0)
    % % %  *.liquid
```

What each folder does

- **`layout/theme.liquid`** — Runs on every page. Contains `<html>`, `<head>` (with `{{ content_for_header }}` which injects Shopify's required scripts), the global `<body>` wrapper, and typically includes header/footer sections.
- **`templates/`** — One file per page *type*. In OS 2.0 these are `.json` files describing which sections appear and in what order. Classic themes use `.liquid` files with inline markup.
- **`sections/`** — Self-contained modules. Each has Liquid markup plus a `{% schema %}` JSON block defining its settings.
- **`snippets/`** — Fragments called by `{% render 'name' %}`. Cannot contain `{% schema %}` or access the section object directly.

- **`assets/`** — Static files. Referenced by filename only: `{{ 'theme.css' | asset_url }}`.
- **`config/settings_schema.json`** — Defines the theme's global settings (colors, typography, social links) shown in **Theme settings** in the admin.
- **`locales/`** — Translation files. Use `{{ 'sections.header.title' | t }}` instead of hardcoding text.

> **KEY INSIGHT:** If you can only remember one thing: **templates choose sections, sections choose snippets, snippets render objects, objects provide data.** Every customization you make fits into this chain.

Takeaways

- Memorize the folder map — it's identical across OS 2.0 themes.
- `.json` templates = section lists; `.liquid` templates = classic markup.
- `assets/` has no Liquid; `snippets/` has Liquid but no schema.
- `config/settings_schema.json` powers the Theme settings panel.

Chapter 3: OS 2.0 JSON Templates vs Classic Liquid Templates

This is the single most important architectural distinction in modern Shopify theming.

Classic templates (pre-OS 2.0)

Classic templates are `.liquid` files that contain the entire page structure:

```
{% section 'main-product' %}
{% section 'related-products' %}
<div class="product-grid">
  {% for product in collection.products %}
    ...full product card markup...
  {% endfor %}
</div>
```

Everything — markup, loops, section calls — lives in one file. Changing the page means editing code.

OS 2.0 JSON templates

OS 2.0 (introduced ~2021, used by Dawn and all current themes) splits the two concerns:

- **`.json` template** — a machine-readable list of which sections appear, in what order, and with what settings.
- **`.liquid` sections** — the actual rendering code, unchanged.

A `templates/product.json` looks like this:

```
{
  "sections": {
    "main": {
      "type": "main-product",
      "blocks": {
        "title": { "type": "title", "settings": {} },
        "price": { "type": "price", "settings": {} },
        "buy_buttons": { "type": "buy-buttons", "settings": {} }
      },
      "block_order": ["title", "price", "buy_buttons"]
    },
    "related": {
      "type": "related-products",
      "settings": { "heading": "You may also like" }
    }
  },
  "order": ["main", "related"]
}
```

```
| Aspect | Classic `.liquid` | OS 2.0 `.json` |
|---|---|---|
| Page structure | Hardcoded in template | JSON section list |
| Merchant editing | Code only | Drag-and-drop theme editor |
```

Section order	Fixed in code	Reorderable in editor
Section settings	Code-defined	JSON + schema
Blocks	Not supported	First-class blocks inside sections
Best for	Legacy themes	All new development

Why JSON templates matter to you

1. **Merchants can reorder sections** without touching code.
2. **You can ship a section once and let users place it anywhere** — product page, homepage, collection page.
3. **The theme editor becomes your design tool** — most customizations never require code edits.
4. **Duplicate templates** (`product.custom.json`) let you create alternate product layouts per product.

> **WARNING:** In an OS 2.0 theme, editing `templates/product.json` in the theme editor changes the *live* template. Always duplicate the theme or work on a dev copy first (Chapter 15).

The `{% section %}` vs `{% sections %}` tag

- Classic: `{% section 'main-product' %}` calls one section by filename.
- OS 2.0 layouts use `{% sections 'header-group' %}` and `{% sections 'footer-group' %}` to render section *groups* defined in `sections/header-group.json`.

Takeaways

- JSON templates = ordered list of sections + settings; sections still write Liquid.
- OS 2.0 is what Dawn and all new themes use — learn it first.
- `{% sections 'group-name' %}` renders grouped sections (header/footer).
- Alternate templates = new `.json` files assigned per product/collection/page.

Chapter 4: Objects Deep Dive — The Data You Can Reach

Objects are Shopify's data model exposed to Liquid. Each page type gives you a different "global" object. Learn the top ones and 90% of theming is solved.

`product` — the most important object

Available on product templates, cart (line items), collection pages (per loop), search results.

Property	Type	Example
`product.title`	string	`"Organic Cotton Tee"`
`product.handle`	string	`"organic-cotton-tee"` — URL slug
`product.id`	number	`9123456789`
`product.price`	number (cents)	`2999` — use `money` filter
`product.compare_at_price`	number	`3999` or `nil`
`product.available`	boolean	`true` / `false`
`product.variants`	array	All variants
`product.selected_or_first_available_variant`	object	Current variant
`product.images`	array	All product images
`product.options`	array	`{ name: "Size", values: [...] }`
`product.description`	HTML string	Rich text
`product.type` / `.vendor` / `.tags`	strings/array	Categorization
`product.url`	string	Relative URL
`product.metafields`	object	Custom data
`product.selling_plan_groups`	array	Subscriptions

`collection` — collections and products within

Property	Notes
`collection.title`, `.handle`, `.description`	Basic info
`collection.products`	**Paginated** — max ~50 per page, use `paginate`
`collection.products_count`	Total products
`collection.all_products_count`	Includes filtered-out
`collection.current_type` / `.current_vendor`	Active filter values
`collection.url`, `.image`	URL and hero image
`collection.metafields`	Custom data

`cart` — the cart page and cart drawer

Property	Notes
`cart.item_count`	Total items

```
| `cart.total_price` | In cents |
| `cart.currency` | `"USD"` etc. |
| `cart.items` | Array of line items |
| `cart.attributes` | Custom cart attributes |
| `item.product`, `item.variant`, `item.quantity` | Per line item |
| `item.final_line_price` | After discounts |
| `item.properties` | Line item custom properties |

### `customer` — logged-in customer data

| Property | Notes |
|---|---|
| `customer` | `nil` if logged out — always guard with `{% if customer %}` |
| `customer.first_name`, `.last_name`, `.email` | Basic info |
| `customer.orders` | Order history (paginated) |
| `customer.addresses` | Saved addresses |
| `customer.tags` | Array of tags |
| `customer.metafields` | Custom data |
| `customer.default_address` | Primary address object |

### `article` and `blog`

- `article.title`, `.content`, `.image`, `.author`, `.published_at`, `.tags`, `.url`
- `blog.articles` — paginated list of articles in a blog
- `blog.all_tags` — for tag filtering on blogs

### `linklists` — navigation menus

{% for link in linklists.main-menu.links %}
  <a href="{{ link.url }}"{% if link.active %} class="active"{% endif %}>
    {{ link.title }}
  </a>
{% endfor %}



| Property | Notes |
|---|---|
| `linklists.handle.links` | Top-level menu items |
| `link.links` | Nested child links |
| `link.active`, `link.child_active` | Current page detection |
| `link.object` | The resource the link points to (if internal) |
| `link.type` | `"product_link"`, `"collection_link"`, `"http"`, `"email"` etc. |

### `images` and `image` — image handling



{{ product.featured_image | image_url: width: 800 | image_tag:
   loading: 'lazy',
   sizes: '(min-width: 750px) 50vw, 100vw',
   widths: '200,400,600,800,1000,1200,1400,1600' }}



| Filter | Purpose |
|---|---|
| `image_url: width: 800` | CDN URL at size |
| `image_tag` | Full `<img>` with srcset |
| `image_url: height: x, crop: 'center'` | Cropped image |
| `image_tag: class: 'foo'` | Add CSS class |

> **PRO TIP:** Always use `image_url` + `image_tag` (or `image_picker`-powered
settings). Never hardcode image URLs — Shopify's CDN needs width/height params to serve
optimized sizes.

### `pages`, `blogs`, `search`, `shop`

- `pages` — all pages, for custom page lists
- `blogs` — all blogs
- `search.results` — search results object
- `shop` — store-wide info: `shop.name`, `shop.currency`, `shop.email`, `shop.metafields`

### Takeaways

- `product`, `collection`, `cart`, `customer`, `article`/`blog`, `linklists` are your
core objects.
- `collection.products` is paginated — always pair with `paginate`.
```

- `customer` is `nil` for guests — guard every access.
- `linklists` handles all navigation; don't hardcode menu URLs.
- `image_url` + `image_tag` is the correct way to render any image.

Chapter 5: Objects Part 2 — `all_products`, `collections`, and the Request Object

Some objects live outside page templates. Master these and you can build global features.

`all_products['handle']` — fetch any product anywhere

```
{% assign featured = all_products['best-seller'] %}
{{ featured.title }} — {{ featured.price | money }}
```

Use for hardcoded "staff picks" or promotional products in any section. Limit: this is a lookup, not a loop — you can't iterate it. For loops, use `collections['handle'].products` or metafields.

`collections['handle']` — fetch any collection

```
{% assign new_arrivals = collections['new-arrivals'] %}
{% for product in new_arrivals.products limit: 8 %}
  {{ product.title }}
{% endfor %}
```

`routes` — URL helpers

```
<a href="{{ routes.cart_url }}">Cart</a>
<a href="{{ routes.account_url }}">Account</a>
<a href="{{ routes.root_url }}">Home</a>
<a href="{{ routes.search_url }}?q=shirt">Search shirts</a>
```

Always use `routes.*` instead of hardcoding `/cart`, `/account` etc. Shopify apps and locales can change URLs.

`request` — current page context

Property	Notes
`request.page_type`	`"product"`, `"collection"`, `"index"`, `"cart"`, `"search"`, `"article"`, `"blog"`, `"page"`, `"password"`, `"policy"`
`request.path`	Current URL path
`request.locale`	Active locale code
`request.design_mode`	`true` when previewing in theme editor

Use `request.page_type` to write context-aware snippets:

```
{% if request.page_type == 'product' %}
  {%- render 'product-schema' -%}
{% elsif request.page_type == 'article' %}
  {%- render 'article-schema' -%}
{% endif %}
```

`content_for_header` and `content_for_layout`

- `{{ content_for_header }}` — **required** in `<head>`. Shopify injects theme-check scripts, app embeds, analytics, and the theme editor's JS bridge. Omitting it breaks the theme editor and many apps.
- `{{ content_for_layout }}` — renders the page's template content. Place inside `<body>` in `layout/theme.liquid`.

`canonical_url`, `page_title`, `page_description`

```
<link rel="canonical" href="{{ canonical_url }}">
<title>{{ page_title }}</title>
<meta name="description" content="{{ page_description | escape }}">
```

Takeaways

- `all_products['handle']` and `collections['handle']` are global lookups.
- `routes.*` is the correct way to link to cart/account/search.
- `request.page_type` lets snippets know what page they're on.
- Never omit `content_for_header` — it powers the editor and apps.
- Use `canonical_url`, `page_title`, `page_description` for SEO.

Chapter 6: Filters — The Transformation Layer

Filters modify output. Syntax: `{{ value | filter_name: arg1, arg2 }}`. They chain left-to-right.

Money and numbers

Filter	Example	Result
`money`	`{{ product.price | money }}`	`$29.99`
`money_with_currency`	`{{ product.price | money_with_currency }}`	`$29.99 USD`
`money_without_currency`	`{{ product.price | money_without_currency }}`	`29.99`
`times: 100`	`{{ 2.5 | times: 100 }}`	`250`
`divided_by: 100`	`{{ 2999 | divided_by: 100 }}`	`29` (integer!)
`round`, `ceil`, `floor`	`{{ 3.7 | round }}`	`4`

> **WARNING:** `divided_by` returns an integer in Liquid. `{{ 2999 | divided_by: 100 }}` gives `29`, not `29.99`. For money, always use `| money` on the raw cents value — never do math on prices manually.

Strings

Filter	Example	Result
`upcase` / `downcase` / `capitalize`	`{{ 'hello' | upcase }}`	`HELLO`
`escape`	`{{ user_input | escape }}`	HTML-escaped
`truncate: 50`	`{{ 'long text' | truncate: 50 }}`	`long text...`
`slice: 0, 3`	`{{ 'abcdef' | slice: 0, 3 }}`	`abc`
`replace: 'a', 'b'`	`{{ 'banana' | replace: 'a', 'o' }}`	`bonono`
`append` / `prepend`	`{{ 'a' | append: 'b' }}`	`ab`
`strip_html`	`{{ '<p>hi</p>' | strip_html }}`	`hi`
`newline_to_br`	`{{ 'a\nb' | newline_to_br }}`	`a b`
`t`	`{{ 'general.continue_shopping' | t }}`	Translation lookup

Arrays

Filter	Example	Result
`size`	`{{ collection.products | size }}`	Count
`first` / `last`	`{{ collection.products | first }}`	First item
`join: ', '`	`{{ product.tags | join: ', ' }}`	`tag1, tag2`
`map: 'title'`	`{{ collection.products | map: 'title' }}`	Array of titles
`where: 'available'`	`{{ collection.products | where: 'available' }}`	Filtered array
`sort` / `sort: 'price'`	`{{ collection.products | sort: 'price' }}`	Sorted
`uniq`	`{{ collection.products | map: 'vendor' | uniq }}`	Unique vendors
`reverse`	`{{ collection.products | reverse }}`	Reversed order
`limit: 4`	`{{ collection.products | limit: 4 }}`	First 4 only

Dates and times

Filter	Example	Result
`date: '%B %d, %Y'`	`{{ article.published_at | date: '%B %d, %Y' }}`	`January 15, 2025`
`date: '%s'`	Unix timestamp	For comparisons

URLs and links

Filter	Example	Result
`link_to`	`{{ product.title | link_to: product.url }}`	`<a href="...">title</a>`

```
| `asset_url` | `{{ 'theme.css' | asset_url }}` | CDN URL for asset |
| `file_url` | `{{ file | file_url }}` | URL for file objects |
| `within` | `{{ product.url | within: collection }}` | URL within collection context |
| `payment_type_svg_tag` | `{{ type | payment_type_svg_tag }}` | Payment icon SVG |
| `font_face` | `{{ settings.type_font | font_face }}` | `@font-face` CSS |

### Images (recap)

| Filter | Purpose |
|---|---|
| `image_url: width: 800` | CDN URL at width |
| `image_url: height: 600, crop: 'center'` | Cropped |
| `image_tag` | Full `<img>` with srcset |
| `img_url` | Legacy — use `image_url` |

### Takeaways

- `| money` always for prices — never manual math on cents.
- `divided_by` truncates to integer; use `times`/`minus` carefully.
- `map`, `where`, `uniq`, `sort` are your array workhorses.
- `escape` any user-generated content.
- `t` for all translatable strings.
- `date` filter with strftime format for all dates.

---

## Chapter 7: Control Flow — `{% if %}`, `{% unless %}`, `{% case %}`, and Loops

### `{% if %} / {% elsif %} / {% else %} / {% endif %}`
```

```
{% if product.available %}
  <button class="add-to-cart">Add to cart</button>
{% elsif product.compare_at_price %}
  <p class="sold-out">Sold out — check back soon</p>
{% else %}
  <p class="unavailable">Unavailable</p>
{% endif %}
```

```
Operators: `==`, `!=`, `>`, `<`, `>=`, `<=`, `contains`, `and`, `or`.

> **WARNING:** Liquid's `and`/`or` don't short-circuit like most languages. `{% if customer and customer.first_name %}` evaluates both sides — `customer.first_name` on a `nil` customer returns `nil` safely, but complex expressions can surprise you. Test conditions carefully.

### `{% unless %}` — inverted if
```

```
{% unless product.available %}
  <p>This product is not available.</p>
{% endunless %}
```

```
### `{% case %}` — multi-way branch
```

```
{% case product.type %}
  {% when 'Shirts' %}
    {%- render 'shirt-details' -%}
  {% when 'Pants' %}
    {%- render 'pants-details' -%}
  {% else %}
    {%- render 'default-details' -%}
{% endcase %}
```

```
### `{% for %}` loops
```

```
{% for product in collection.products %}
  <div class="product-card">{{ product.title }}</div>
{% endfor %}
```

Loop properties: `forloop.index`, `forloop.first`, `forloop.last`, `forloop.length`, `forloop.index0`.

```
{% for product in collection.products limit: 4 %}
  {{ product.title }}{% unless forloop.last %}, {% endunless %}
{% endfor %}
```

> **KEY INSIGHT:** `collection.products` returns at most ~50 products per page. If you need more, use `{% paginate %}` (Chapter 8) or filter with `where`/`limit`.

`{% assign %}` and `{% capture %}`

```
{% assign sale_price = product.price | times: 0.8 %}
{% capture image_html %}
  {{ product.featured_image | image_url: width: 400 | image_tag }}
{% endcapture %}
{{ image_html }}
```

`assign` stores a value; `capture` stores rendered output (multi-line).

`{% cycle %}` — alternating values

```
{% for product in collection.products %}
  <div class="card {% cycle 'even', 'odd' %}">...</div>
{% endfor %}
```

`{% comment %}` — comments

```
{% comment %} This won't render. Use for notes. {% endcomment %}
```

`{% raw %}` — literal Liquid

```
{% raw %}{{ product.title }}{% endraw %}
<!-- Outputs literally: {{ product.title }} -->
```

Takeaways

- `if/elsif/else/endif` for conditions; `case/when/else/endcase` for multi-branch.
- `for` loops have `forloop.*` properties for index/first/last.
- `assign` for values, `capture` for rendered blocks.
- `cycle` for alternating classes without JS.
- `comment` and `raw` are your documentation and escape hatches.

Chapter 8: Pagination and Performance Loops

`{% paginate %}`

```
{% paginate collection.products by 24 %}
  {% for product in collection.products %}
    ...product card...
  {% endfor %}

  {% render 'pagination', paginate: paginate %}
{% endpaginate %}
```

Inside `paginate`, you get a `paginate` object:

Property	Notes
`paginate.pages`	Total pages
`paginate.current_page`	Current page number
`paginate.previous`	URL to prev page (or nil)
`paginate.next`	URL to next page (or nil)
`paginate.parts`	Array of page links for rendering

Render pagination links:

```
<nav class="pagination">
  {% if paginate.previous %}
    <a href="{{ paginate.previous.url }}" class="prev">!• Prev</a>
  {% endif %}
  {% for part in paginate.parts %}
    {% if part.is_link %}
      <a href="{{ part.url }}">{{ part.title }}</a>
    {% else %}
      <span class="current">{{ part.title }}</span>
    {% endif %}
  {% endfor %}
  {% if paginate.next %}
    <a href="{{ paginate.next.url }}" class="next">Next !’</a>
  {% endif %}
</nav>
```

Performance rules

1. **Never nest `for` loops over large collections.** `collection.products` × `collection.products` = O(n²) and will timeout or freeze the page.
2. **Always `limit` your loops** unless paginating.
3. **Use `paginate` for anything over ~50 items.**
4. **Don't loop `collections.all` or `blogs.all` in production** — these are unbounded.
5. **Cache expensive lookups** — `all_products['handle']` is a lookup, but repeated lookups in a loop are wasteful. Assign once.
6. **Use `| where` before `| limit`** to reduce data before limiting.

```
{%- comment -%} GOOD: filter then limit {%- endcomment -%}
{% assign available = collection.products | where: 'available' | limit: 8 %}
{% for product in available %}...{% endfor %}

{%- comment -%} BAD: loop all, check inside {%- endcomment -%}
{% for product in collection.products %}
  {% if product.available %}...{% endif %}
{% endfor %}
```

Takeaways

- `{% paginate by N %}` controls page size and gives you `paginate.*` object.
- `paginate.parts` is an array — loop it for page number links.
- Filter with `where`/`limit` before looping.
- Never nest large loops; never iterate unbounded objects.

Chapter 9: Sections and Blocks — The Schema System

Every section file ends with a `{% schema %}` JSON block. This is what makes the theme editor work.

Anatomy of a section

```
<div class="section-{{ section.id }}" {{ section.shopify_attributes }}>
  {% if section.settings.heading != blank %}
    <h2>{{ section.settings.heading }}</h2>
  {% endif %}
```

```
  <div class="grid">
    {% for block in section.blocks %}
      {% case block.type %}
        {% when 'product-card' %}
          {%- render 'product-card', product: block.settings.product, block: block -%}
        {% when 'text' %}
          <div class="text-block" {{ block.shopify_attributes }}>
            {{ block.settings.text }}
          </div>
      {% endcase %}
    {% endfor %}
  </div>
</div>

{% schema %}
{
  "name": "Product grid",
  "settings": [
    {
      "type": "text",
      "id": "heading",
      "label": "Heading",
      "default": "Featured products"
    },
    {
      "type": "range",
      "id": "columns",
      "label": "Columns",
      "min": 2, "max": 5, "step": 1, "default": 4
    },
    {
      "type": "color",
      "id": "background",
      "label": "Background",
      "default": "#ffffff"
    },
    {
      "type": "checkbox",
      "id": "show_vendor",
      "label": "Show vendor",
      "default": false
    }
  ],
  "blocks": [
    {
      "type": "product-card",
      "name": "Product card",
      "settings": [
        { "type": "product", "id": "product", "label": "Product" }
      ]
    },
    {
      "type": "text",
      "name": "Text",
      "settings": [
        { "type": "richtext", "id": "text", "label": "Text" }
      ]
    }
  ],
  "presets": [
    {
      "name": "Product grid",
      "blocks": [
        { "type": "product-card" },
        { "type": "product-card" }
      ]
    }
  ]
}
{% endschema %}
```

```
### Setting types

| Type | Editor control | Value |
|---|---|---|
| `text` | Single-line input | string |
| `textarea` | Multi-line | string |
```

```
| `richtext` | WYSIWYG editor | HTML string |
| `number` | Numeric input | number |
| `range` | Slider | number |
| `checkbox` | Toggle | boolean |
| `select` | Dropdown | string (from `options`) |
| `radio` | Radio buttons | string |
| `color` | Color picker | hex string |
| `color_background` | Color + gradient picker | object |
| `image_picker` | Image selector | image object |
| `product` | Product picker | product object |
| `collection` | Collection picker | collection object |
| `page` | Page picker | page object |
| `blog` / `article` | Blog/article picker | object |
| `url` | Link picker | URL string |
| `font_picker` | Font selector | font object |
| `liquid` | Code editor | raw Liquid string |
| `html` | HTML editor | HTML string |
| `video` | Video picker | video object |
| `video_url` | Video URL | string |

### `section` object inside sections

| Property | Notes |
|---|---|
| `section.id` | Unique ID — use for CSS scoping |
| `section.settings.*` | All settings values |
| `section.blocks` | Array of block objects |
| `section.blocks.size` | Block count |
| `section.shopify_attributes` | Required for editor drag-and-drop |
| `section.index` | Position index |
| `section.index | plus: 1` | 1-based index |

### `block` object inside sections

| Property | Notes |
|---|---|
| `block.id` | Unique ID |
| `block.type` | Block type name |
| `block.settings.*` | Block settings |
| `block.shopify_attributes` | Required for editor |

### `presets` — theme editor "Add section"
```

```
"presets": [
  {
    "name": "FAQ",
    "category": "Text",
    "settings": { "heading": "Frequently asked questions" },
    "blocks": [
      { "type": "question", "settings": { "question": "What is your return policy?" } }
    ]
  }
]
```

```
Presets appear in the theme editor's "Add section" menu. Without presets, your section
won't be addable via UI (only via JSON).

### `enabled_on` / `disabled_on` — template targeting
```

```
"enabled_on": { "templates": ["product", "collection"] }
"disabled_on": { "templates": ["cart", "password"] }
```

```
Controls which template types can use the section.

### Takeaways

- Every section ends with `{% schema %}...{% endschema %}` containing JSON.
- `section.settings`, `section.blocks`, `section.id`, `section.shopify_attributes` are
your interface.
- `presets` make sections addable in the theme editor.
- `enabled_on`/`disabled_on` control template targeting.
- `shopify_attributes` is required on sections and blocks for editor drag-and-drop.
```

Chapter 10: Blocks — Reusable Content Units

Blocks are the OS 2.0 answer to "I want a section with customizable inner pieces." A section can contain blocks of different types; merchants add/remove/reorder them in the editor.

Static vs dynamic blocks

- **Static blocks** — defined in the template JSON, always present, can't be removed (e.g., `title`, `price`, `buy_buttons` on product pages).
- **Dynamic blocks** — merchants add/remove freely (e.g., custom text, image, product card).

Block types in Dawn's product section

Block type	Purpose
`title`	Product title
`price`	Price display
`sku`	SKU text
`vendor`	Vendor name
`variant_picker`	Size/color selector
`buy_buttons`	Add to cart + dynamic checkout
`description`	Product description
`share`	Social share buttons
`custom_liquid`	Raw Liquid/HTML block
`collapsible_tab`	FAQ accordion item
`popup`	Modal trigger
`rating`	Review stars
`complementary`	"Pairs well with" products

Rendering blocks

```
{% for block in section.blocks %}
  {% case block.type %}
    {% when 'title' %}
      <h1 class="product-title" {{ block.shopify_attributes }}>
        {{ product.title }}
      </h1>
    {% when 'price' %}
      <div class="product-price" {{ block.shopify_attributes }}>
        {%- render 'price', product: product, use_variant: true -%}
      </div>
    {% when 'buy_buttons' %}
      <div class="buy-buttons" {{ block.shopify_attributes }}>
        {%- render 'buy-buttons', product: product, block: block -%}
      </div>
    {% when 'custom_liquid' %}
      <div {{ block.shopify_attributes }}>
        {{ block.settings.custom_liquid }}
      </div>
  {% endcase %}
{% endfor %}
```

Block settings in schema

```
"blocks": [
  {
    "type": "collapsible_tab",
    "name": "Collapsible row",
    "settings": [
      { "type": "text", "id": "heading", "label": "Heading" },
      { "type": "richtext", "id": "content", "label": "Content" },
      { "type": "checkbox", "id": "open_by_default", "label": "Open by default" }
    ]
  }
]
```

Accessing block settings

```
{{ block.settings.heading }}
{{ block.settings.content }}
{% if block.settings.open_by_default %}open{% endif %}



### Takeaways

- Blocks are inner components of sections; merchants reorder/add/remove them.
- Static blocks are template-defined; dynamic blocks are merchant-managed.
- Render blocks with `{% for block in section.blocks %}` + `{% case block.type %}`.
- Always include `{{ block.shopify_attributes }}` for editor support.
- Define block types and their settings in the section's `schema.blocks` array.

---

## Chapter 11: Snippets vs Sections vs App Blocks — When to Use What

### Comparison table

| Feature | Snippet | Section | App block |
|---|---|---|---|
| File location | `snippets/` | `sections/` | `blocks/` (OS 2.0) |
| Has `{% schema %}` | No | Yes | Yes |
| Appears in theme editor | No | Yes | Yes (as section block) |
| Can be placed anywhere | Only where `{% render %}` called | Only in templates | Only
inside parent section |
| Receives parameters | Via `{% render 'x', param: value %}` | Via `section` object |
Via `block` object |
| Merchant editable | No | Yes | Yes |
| Access to `section` | No (unless passed) | Yes | Via parent |
| Use case | Reusable markup fragments | Page modules | Inner section components |

### Snippets — the workhorse



{%- comment -%} snippets/product-card.liquid {%- endcomment -%}
{% assign product = product_card_product %}
<div class="card">
  <a href="{{ product.url }}">
    {{ product.featured_image | image_url: width: 400 | image_tag }}
    <h3>{{ product.title }}</h3>
    <p>{{ product.price | money }}</p>
  </a>
</div>



Called via:



{% render 'product-card', product_card_product: product %}
```

> **PRO TIP:** Always use `{%- render -%}` with hyphens to strip whitespace. In loops, this prevents hundreds of blank lines in your HTML and speeds up rendering.

Sections — the page builder

Sections are the only thing merchants can add/reorder in the theme editor. Every section needs a `schema` with `presets` to appear in "Add section."

App blocks (OS 2.0)

App blocks are sections in `blocks/` directory that apps inject. They appear inside other sections' block lists. You rarely create these manually — apps do.

Decision guide

```
Need reusable markup?           !’ Snippet
Need merchant-editable module?  !’ Section
Need inner piece of a section?  !’ Block (inside section)
Need app-injectable content?    !’ App block
```

Takeaways

- Snippets: no schema, no editor, render via `{% render %}`.
- Sections: schema + presets, editor-manageable, one per template slot.
- Blocks: inner components of sections, defined in schema.
- App blocks: for apps, not manual creation.
- Use `{%- render -%}` to avoid whitespace bloat.

Chapter 12: The Theme Editor and Custom Liquid Blocks

What the theme editor does

The theme editor (Online Store !’ Themes !’ Customize) lets merchants:

- Reorder sections on any template
- Edit section settings (text, colors, images, products)
- Add/remove/reorder blocks within sections
- Edit theme-wide settings (colors, fonts, social links)
- Preview changes before publishing

Adding a section in the editor

1. Go to **Online Store !’ Themes !’ Customize**.
2. Select the template you want to edit (e.g., Product pages).
3. Click **Add section** in the left sidebar.
4. Find your section by name (from `presets` in schema).
5. Drag it to the desired position.
6. Click the section to edit its settings.
7. Click **Save**.

Adding a block inside a section

1. In the theme editor, click the section that contains blocks.
2. In the sidebar, find the **Blocks** section.
3. Click **Add block**.
4. Choose a block type (e.g., "Text", "Image", "Product card").
5. Configure the block's settings.
6. Drag blocks to reorder.
7. Click **Save**.

Custom Liquid blocks

Dawn includes a `custom-liquid` block type that lets merchants paste raw HTML/Liquid into any section that supports blocks. To enable it:

1. In the theme editor, go to the section you want to modify.
2. Click **Add block** !’ **Custom Liquid**.
3. Paste your Liquid/HTML code into the block's text area.
4. Save.

Example custom Liquid block content:

```
<div class="trust-badges">
  <span>'  Free shipping over $50</span>
  <span>'  30-day returns</span>
  <span>'  Secure checkout</span>
</div>
```

> **WARNING:** Custom Liquid blocks run with full Liquid access but **no section context**. You can't access `section.settings` inside a custom liquid block — only global objects like `product`, `collection`, `shop`. For section-specific data, build a proper block type in schema.

Theme editor limitations

- Can't edit `layout/theme.liquid` directly (except via code editor).
- Can't create new sections — only use existing ones with presets.
- Can't edit `assets/` files — only via code editor.
- JSON templates can be edited as raw JSON in the code editor, but the visual editor is safer.

Takeaways

- Theme editor = visual section/block/settings management.
- "Add section" requires `presets` in schema.
- "Add block" requires block types in schema.
- Custom Liquid blocks allow raw code but lack section context.
- Use the code editor for `layout/`, `assets/`, and `config/` files.

Chapter 13: CSS and JavaScript in Assets

How assets work

Files in `assets/` are served statically. Reference them in Liquid:

```
{{ 'theme.css' | asset_url | stylesheet_tag }}
{{ 'theme.js' | asset_url | script_tag }}
{{ 'logo.png' | asset_url | image_tag }}
```

Or with parameters:

```
<link rel="stylesheet" href="{{ 'custom.css' | asset_url }}?v={{ shop.time | date:
'%s' }}">
```

CSS architecture in Dawn

Dawn uses a single compiled `theme.css` (or `base.css` + component files). Key patterns:

```
/* CSS custom properties for theming */
:root {
  --color-background: {{ settings.colors_background }};
  --color-foreground: {{ settings.colors_text }};
}

/* Component-scoped styles */
.product-card { ... }
.product-card__image { ... }
.product-card__title { ... }
```

> **KEY INSIGHT:** Dawn's CSS uses BEM naming (`block__element--modifier`). Follow this convention for maintainability. `.product-card__image` not `.productImage`.

Adding custom CSS

Option 1: Theme settings !’ Custom CSS (if theme supports it)

Option 2: Custom CSS file

1. Create `assets/custom.css`.
2. Add to `layout/theme.liquid` before `</head>`:

```
{{ 'custom.css' | asset_url | stylesheet_tag }}
```

Option 3: Inline in section (for section-specific styles)

```
<style>
  .section-{{ section.id }} .custom-class { color: red; }
</style>
```

Adding custom JavaScript

Option 1: Custom JS file

1. Create `assets/custom.js`.
2. Add to `layout/theme.liquid` before `</body>`:

```
{{ 'custom.js' | asset_url | script_tag }}
```

Option 2: Section-specific JS

```
<script>
  document.addEventListener('DOMContentLoaded', function() {
    // Your code here
  });
</script>
```

JavaScript best practices

- Wrap in `DOMContentLoaded` or use `defer`.
- Use event delegation for dynamic content.
- Respect `prefers-reduced-motion`.
- Test on mobile — touch events differ.
- Don't conflict with theme's existing JS (check `theme.js`).

Takeaways

- `assets/` files are static; reference via `asset_url`.
- Dawn uses BEM CSS naming — follow it.
- Custom CSS: theme settings, custom file, or inline `<style>`.
- Custom JS: custom file or inline `<script>` with `DOMContentLoaded`.
- Always scope section CSS with `section.id` to avoid conflicts.

Chapter 14: Customization Recipes — Six Production-Ready Patterns

Recipe 1: Announcement bar

Create `sections/announcement-bar.liquid`:

```
{% if section.settings.enabled %}
  <div class="announcement-bar" style="background: {{ section.settings.bg_color }};
color: {{ section.settings.text_color }};">
    <p>{{ section.settings.message }}</p>
    {% if section.settings.link_text != blank %}
      <a href="{{ section.settings.link }}">{{ section.settings.link_text }}</a>
    {% endif %}
  </div>
{% endif %}

{% schema %}
{
  "name": "Announcement bar",
  "settings": [
    { "type": "checkbox", "id": "enabled", "label": "Enabled", "default": true },
    { "type": "text", "id": "message", "label": "Message", "default": "Free shipping on
orders over $50" },
    { "type": "url", "id": "link", "label": "Link URL" },
    { "type": "text", "id": "link_text", "label": "Link text", "default": "Shop now" },
    { "type": "color", "id": "bg_color", "label": "Background", "default": "#000000" },
    { "type": "color", "id": "text_color", "label": "Text", "default": "#ffffff" }
  ],
  "presets": [{ "name": "Announcement bar" }]
}
{% endschema %}
```

Add to `templates/index.json` or any template via theme editor.

Recipe 2: Product page trust badges

Add a block to `sections/main-product.liquid` schema:

```
{
  "type": "trust_badges",
  "name": "Trust badges",
  "settings": [
    { "type": "text", "id": "badge1", "label": "Badge 1", "default": "Free shipping" },
    { "type": "text", "id": "badge2", "label": "Badge 2", "default": "30-day returns" },
    { "type": "text", "id": "badge3", "label": "Badge 3", "default": "Secure checkout" }
  ]
}
```

Render in the section:

```
{% when 'trust_badges' %}
  <div class="trust-badges" {{ block.shopify_attributes }}>
    <span>{{ block.settings.badge1 }}</span>
    <span>{{ block.settings.badge2 }}</span>
    <span>{{ block.settings.badge3 }}</span>
  </div>
```

Recipe 3: Cart upsell

Create `sections/cart-upsell.liquid`:

```
{% if section.settings.products != blank %}
  <div class="cart-upsell">
    <h3>{{ section.settings.heading }}</h3>
    <div class="upsell-grid">
      {% for product in section.settings.products %}
        <div class="upsell-card">
          {{ product.featured_image | image_url: width: 200 | image_tag }}
          <h4>{{ product.title }}</h4>
          <p>{{ product.price | money }}</p>
          <form method="post" action="{{ routes.cart_add_url }}">
            <input type="hidden" name="id"
value="{{ product.selected_or_first_available_variant.id }}">
            <button type="submit">Add to cart</button>
          </form>
        </div>
      {% endfor %}
    </div>
  </div>
{% endif %}

{% schema %}
{
  "name": "Cart upsell",
  "settings": [
    { "type": "text", "id": "heading", "label": "Heading", "default": "You might also
like" },
    { "type": "product_list", "id": "products", "label": "Products", "limit": 4 }
  ],
  "presets": [{ "name": "Cart upsell" }]
}
{% endschema %}
```

Add to `templates/cart.json`.

Recipe 4: Collection filters (OS 2.0)

OS 2.0 themes handle filtering via URL parameters. Create a custom filter section:

```
{% assign current_tags = collection.all_tags | where: 'available' %}
<div class="filters">
  <h4>Filter by tag</h4>
```

```
  {% for tag in current_tags %}
    {% if collection.current_tags contains tag %}
      <a href="{{ collection.url }}/{{ tag | handleize }}" class="active">{{ tag }}</a>
    {% else %}
      <a href="{{ collection.url }}/{{ tag | handleize }}">{{ tag }}</a>
    {% endif %}
  {% endfor %}
</div>
```

````
For price/vendor filters, use Shopify's native filtering (search & discovery app) or
build custom logic with `collection.current_type` and `collection.current_vendor`.

### Recipe 5: FAQ accordion

Create `sections/faq.liquid`:
````

```
<div class="faq-section">
  <h2>{{ section.settings.heading }}</h2>
  <div class="faq-list">
    {% for block in section.blocks %}
      <details class="faq-item" {{ block.shopify_attributes }}>
        <summary>{{ block.settings.question }}</summary>
        <div class="faq-answer">{{ block.settings.answer }}</div>
      </details>
    {% endfor %}
  </div>
</div>

{% schema %}
{
  "name": "FAQ",
  "settings": [
    { "type": "text", "id": "heading", "label": "Heading", "default": "Frequently asked
questions" }
  ],
  "blocks": [
    {
      "type": "question",
      "name": "Question",
      "settings": [
        { "type": "text", "id": "question", "label": "Question" },
        { "type": "richtext", "id": "answer", "label": "Answer" }
      ]
    }
  ],
  "presets": [
    {
      "name": "FAQ",
      "blocks": [
        { "type": "question", "settings": { "question": "What is your return
policy?" } },
        { "type": "question", "settings": { "question": "How long does shipping
take?" } }
      ]
    }
  ]
}
{% endschema %}
```

````
### Recipe 6: Sticky add-to-cart

Create `sections/sticky-atc.liquid`:
````

```
{% if template == 'product' %}
  <div class="sticky-atc" id="sticky-atc">
    <div class="sticky-atc-info">
      <img src="{{ product.featured_image | image_url: width: 100 }}"
alt="{{ product.title }}">
      <div>
        <h4>{{ product.title }}</h4>
        <p>{{ product.price | money }}</p>
      </div>
```

```
    </div>
    <button class="sticky-atc-button"
onclick="document.querySelector('form[action*=\'cart\']
button[type=\'submit\']').click()">
      Add to cart
    </button>
  </div>
{% endif %}

{% schema %}
{
  "name": "Sticky add to cart",
  "settings": [
    { "type": "checkbox", "id": "enabled", "label": "Enabled", "default": true }
  ],
  "presets": [{ "name": "Sticky add to cart" }]
}
{% endschema %}
```

Add CSS:

```
.sticky-atc {
  position: fixed;
  bottom: 0;
  left: 0;
  right: 0;
  background: #fff;
  padding: 1rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
  box-shadow: 0 -2px 10px rgba(0,0,0,0.1);
  transform: translateY(100%);
  transition: transform 0.3s;
}
.sticky-atc.visible { transform: translateY(0); }
```

Add JS:

```
document.addEventListener('DOMContentLoaded', function() {
  const sticky = document.getElementById('sticky-atc');
  const mainAtc = document.querySelector('form[action*="cart"]');
  if (!sticky || !mainAtc) return;

  const observer = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
      if (!entry.isIntersecting) sticky.classList.add('visible');
      else sticky.classList.remove('visible');
    });
  });
  observer.observe(mainAtc);
});
```

Takeaways

- Announcement bar: simple section with text/color/link settings.
- Trust badges: block type in product section schema.
- Cart upsell: section with `product_list` setting, added to cart template.
- Filters: use URL params + `collection.current_*` or Search & Discovery app.
- FAQ: details/summary HTML + blocks for questions.
- Sticky ATC: fixed-position div + IntersectionObserver JS.

Chapter 15: Debugging Liquid — Errors, Mistakes, and Fixes

Where errors appear

Location	What you see

```
| Theme preview | Red error banner with message |
| Storefront | Blank page or "Liquid error" text |
| Theme editor | Error in section preview |
| Browser console | JS errors (not Liquid) |
| Shopify admin !’ Themes | "Theme has errors" badge |

### Common Liquid errors

| Error | Cause | Fix |
|---|---|---|
| `Liquid syntax error: Unknown tag 'xyz'` | Typo in tag name | Check spelling: `endif`, `endfor`, `endschema` |
| `Liquid syntax error: Expected endtag` | Missing closing tag | Add `{% endif %}`, `{% endfor %}`, etc. |
| `undefined variable` | Accessing property on `nil` | Guard with `{% if object %}` |
| `Liquid error: internal` | Infinite loop or timeout | Reduce loop size, add `limit` |
| `Cannot render` | Section/snippet not found | Check filename matches `{% render 'name' %}` |
| `schema must be valid JSON` | Trailing comma or comment in schema | Validate JSON at jsonlint.com |

### Debugging workflow

1. **Check the theme editor preview** — errors show inline.
2. **Isolate the section** — temporarily remove sections to find the culprit.
3. **Comment out blocks** — use `{% comment %}...{% endcomment %}` to bisect.
4. **Validate schema JSON** — copy schema to jsonlint.com.
5. **Check `content_for_header`** — missing this breaks everything.
6. **Test on a dev store** — never debug on live.

### Common mistakes

{%- comment -%} WRONG: missing endif {%- endcomment -%}
{% if product.available %}
  <p>Available</p>

{%- comment -%} RIGHT {%- endcomment -%}
{% if product.available %}
  <p>Available</p>
{% endif %}



{%- comment -%} WRONG: accessing customer without guard {%- endcomment -%}
<p>{{ customer.first_name }}</p>

{%- comment -%} RIGHT {%- endcomment -%}
{% if customer %}
  <p>{{ customer.first_name }}</p>
{% endif %}



{%- comment -%} WRONG: loop without limit on large collection {%- endcomment -%}
{% for product in collection.products %}
  {{ product.title }}
{% endfor %}

{%- comment -%} RIGHT: use paginate or limit {%- endcomment -%}
{% paginate collection.products by 24 %}
  {% for product in collection.products %}
    {{ product.title }}
  {% endfor %}
{% endpaginate %}


### Debugging tools

- **Shopify Theme Inspector** (Chrome extension) — inspect sections/blocks in editor.
- **Liquid playground** — test snippets at `shopify.github.io/liquid/`.
- **Browser DevTools** — check rendered HTML, not Liquid source.
- **`{{ 'debug' | debug }}`** — outputs variable in page (remove before launch).
```

Takeaways

- Errors show in theme preview/editor as red banners.
- Most errors: missing end tag, nil object access, invalid JSON schema.
- Always guard `customer` and other nullable objects.
- Use `{% comment %}` to bisect problems.
- Validate schema JSON separately.
- Test on dev store, never live.

Chapter 16: Performance Rules — Speed Matters

The performance budget

Shopify pages should load in under 3 seconds. Liquid rendering is fast; the bottleneck is usually:

1. **Too many sections** — each adds render time.
2. **Large loops** — `collection.products` × 50 items × complex card = slow.
3. **Unoptimized images** — always use `image_url` with width.
4. **Heavy JS** — defer non-critical scripts.
5. **External apps** — each app adds scripts/CSS.

Liquid performance rules

Rule	Why
Always `limit` loops	Prevents rendering 1000+ items
Use `paginate` for collections	Splits into pages
Filter before loop	`where`/`limit` reduces data
Avoid nested loops	O(n²) = timeout
Use `assign` once	Don't recompute in loop
Prefer `render` over `include`	`render` isolates scope, faster
Use `{%- -%}` whitespace control	Reduces HTML size

Image performance

```
{%- comment -%} GOOD: responsive image with srcset {%- endcomment -%}
{{ product.featured_image | image_url: width: 800 | image_tag:
   loading: 'lazy',
   sizes: '(min-width: 750px) 50vw, 100vw',
   widths: '200,400,600,800,1000,1200' }}

{%- comment -%} BAD: full-size image {%- endcomment -%}
<img src="{{ product.featured_image | img_url: 'master' }}">
```

CSS/JS performance

- **Defer non-critical JS**: `<script src="..." defer>`.
- **Inline critical CSS**: above-the-fold styles inline.
- **Lazy load images**: `loading="lazy"`.
- **Minimize HTTP requests**: combine CSS files if possible.
- **Use `preconnect`** for external domains.

Section performance

- **Don't render hidden sections**: use `{% if section.settings.enabled %}`.
- **Lazy load below-fold sections**: use `loading="lazy"` on images.
- **Limit block counts**: `section.blocks | size` check.

Takeaways

- Limit all loops; paginate collections.
- Filter before iterating.
- Optimize images with `image_url` + `image_tag`.
- Defer JS, lazy load images.
- Conditional-render hidden sections.
- Test with Shopify's speed score and PageSpeed Insights.

Chapter 17: Testing on a Dev Store and Going Live Safely

Setting up a development store

1. Go to **Shopify Partners** !’ **Stores** !’ **Add store** !’ **Development store**.
2. Name it (e.g., "My Theme Dev").
3. Select a plan (development stores are free).
4. Install your theme.
5. Add test products/collections.
6. Configure payment/test gateways.

Development workflow

1. **Work on dev store** — never edit live theme directly.
2. **Duplicate theme** — Online Store !’ Themes !’ "ï !’ Duplicate.
3. **Edit duplicate** — customize freely.
4. **Preview** — test on dev store.
5. **Push to live** — when ready, publish or transfer.

Safe deployment checklist

- [ ] All changes tested on dev store
- [ ] No Liquid errors in theme preview
- [ ] Mobile responsive (test on real device)
- [ ] Forms work (contact, newsletter, checkout)
- [ ] Images load correctly
- [ ] Apps still function
- [ ] SEO meta tags present
- [ ] `content_for_header` intact
- [ ] Backup taken (duplicate theme)
- [ ] Staging URL tested

Going live

1. **Duplicate live theme** as backup.
2. **Apply changes** to a copy.
3. **Test thoroughly** on the copy.
4. **Publish** the tested copy when confident.
5. **Monitor** for errors after publish.
6. **Keep backup** for 30 days.

Rollback plan

If something breaks:

1. Go to **Online Store !’ Themes**.
2. Find your backup (unpublished duplicate).
3. Click **Publish** on the backup.
4. Investigate and fix on dev store.

Takeaways

- Always test on dev store or theme duplicate.
- Never edit live theme directly.
- Keep a backup theme for rollback.
- Test forms, apps, mobile, SEO before publishing.
- Monitor after publish; rollback if issues.

Chapter 18: Capstone — Building a Custom Product Page Section

Let's build a complete custom product page section from scratch. This is a "Trust & Features" section that displays product-specific trust badges and features.

Step 1: Create the section file

Create `sections/product-trust.liquid`:

```
<div class="product-trust-section" {{ section.shopify_attributes }}>
  <div class="trust-header">
    <h3>{{ section.settings.heading }}</h3>
    {% if section.settings.subheading != blank %}
      <p class="trust-subheading">{{ section.settings.subheading }}</p>
    {% endif %}
  </div>

  <div class="trust-grid" style="grid-template-columns:
repeat({{ section.settings.columns }}, 1fr);">
    {% for block in section.blocks %}
```

```
      <div class="trust-item" {{ block.shopify_attributes }}>
        {% if block.settings.icon != blank %}
          <div class="trust-icon">{{ block.settings.icon }}</div>
        {% endif %}
        <h4 class="trust-title">{{ block.settings.title }}</h4>
        <p class="trust-description">{{ block.settings.description }}</p>
      </div>
    {% endfor %}
  </div>
</div>

{% schema %}
{
  "name": "Product trust",
  "settings": [
    {
      "type": "text",
      "id": "heading",
      "label": "Heading",
      "default": "Why you'll love this"
    },
    {
      "type": "textarea",
      "id": "subheading",
      "label": "Subheading",
      "default": "Our commitment to quality"
    },
    {
      "type": "range",
      "id": "columns",
      "label": "Columns",
      "min": 2,
      "max": 4,
      "step": 1,
      "default": 3
    }
  ],
  "blocks": [
    {
      "type": "trust_item",
      "name": "Trust item",
      "settings": [
        {
          "type": "text",
          "id": "icon",
          "label": "Icon (emoji or SVG)",
          "default": "' "
        },
        {
          "type": "text",
          "id": "title",
          "label": "Title",
          "default": "Premium quality"
        },
        {
          "type": "textarea",
          "id": "description",
          "label": "Description",
          "default": "Made with the finest materials"
        }
      ]
    }
  ],
  "presets": [
    {
      "name": "Product trust",
      "blocks": [
        {
          "type": "trust_item",
          "settings": {
            "icon": "Ø=Þš",
            "title": "Free shipping",
            "description": "On orders over $50"
          }
        },
        {
          "type": "trust_item",
          "settings": {
            "icon": "!©þ ",
```

```
            "title": "Easy returns",
            "description": "30-day return policy"
          }
        },
        {
          "type": "trust_item",
          "settings": {
            "icon": "Ø=Ý ",
            "title": "Secure checkout",
            "description": "Your data is protected"
          }
        }
      ]
    }
  ]
}
{% endschema %}
```

Step 2: Add CSS

Create `assets/product-trust.css`:

```
.product-trust-section {
  padding: 4rem 0;
  background: var(--color-background-secondary, #f9f9f9);
}

.trust-header {
  text-align: center;
  margin-bottom: 3rem;
}

.trust-header h3 {
  font-size: 2rem;
  margin-bottom: 0.5rem;
}

.trust-subheading {
  color: #666;
  font-size: 1.1rem;
}

.trust-grid {
  display: grid;
  gap: 2rem;
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 1rem;
}

.trust-item {
  text-align: center;
  padding: 2rem;
  background: #fff;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
}

.trust-icon {
  font-size: 2.5rem;
  margin-bottom: 1rem;
}

.trust-title {
  font-size: 1.2rem;
  margin-bottom: 0.5rem;
  font-weight: 600;
}

.trust-description {
  color: #666;
  font-size: 0.95rem;
  line-height: 1.5;
}
```

```
@media (max-width: 768px) {
  .trust-grid {
    grid-template-columns: 1fr !important;
  }
}
```

```
### Step 3: Load CSS in the section

Add to top of `product-trust.liquid`:


{{ 'product-trust.css' | asset_url | stylesheet_tag }}
```

```
### Step 4: Add to product template

1. Go to **Online Store !’ Themes !’ Customize**.
2. Select **Product pages** template.
3. Click **Add section** !’ **Product trust**.
4. Drag it below the main product section.
5. Configure heading, subheading, columns.
6. Add/edit trust items (blocks).
7. Click **Save**.

### Step 5: Test and refine

- Preview on desktop and mobile.
- Test with different column counts.
- Add more trust items.
- Adjust colors to match brand.

### What you built

- A reusable section with schema-driven settings.
- Dynamic blocks merchants can add/remove/reorder.
- Responsive CSS with mobile breakpoint.
- Theme editor integration via presets.
- No hardcoded content — all merchant-editable.

> **KEY INSIGHT:** This section could now be added to ANY template — homepage,
collection, about page — without code changes. That's the power of OS 2.0 sections.

### Takeaways

- Sections = Liquid markup + schema + presets.
- Blocks = inner components merchants control.
- CSS in assets, loaded via `asset_url`.
- Theme editor integration is automatic with schema.
- Reusable across all templates.

---

## Chapter 19: Pre-Launch Checklist

Before publishing any theme changes, verify:

### Functionality

- [ ] All pages load without Liquid errors
- [ ] Product pages: variant picker, add to cart, price updates work
- [ ] Collection pages: filters, pagination, sorting work
- [ ] Cart: add/remove/update quantities, totals correct
- [ ] Search: returns relevant results
- [ ] Customer accounts: login, order history work
- [ ] Contact/newsletter forms submit successfully
- [ ] Checkout flow completes (test mode)

### Design

- [ ] Mobile responsive (test on real device)
- [ ] Images load at correct sizes
- [ ] Typography consistent across pages
- [ ] Colors match brand guidelines
- [ ] No layout shifts or broken spacing
- [ ] Hover/focus states work
```

- [ ] Loading states for dynamic content

Technical

- [ ] `content_for_header` present in `layout/theme.liquid`
- [ ] No console errors in browser DevTools
- [ ] No Liquid errors in theme preview
- [ ] Schema JSON validates (no trailing commas)
- [ ] All `routes.*` used instead of hardcoded URLs
- [ ] `customer` object guarded with `{% if customer %}`
- [ ] Images use `image_url` + `image_tag` (not hardcoded)
- [ ] Custom CSS/JS doesn't conflict with theme

SEO

- [ ] Page titles unique and descriptive
- [ ] Meta descriptions present
- [ ] Canonical URLs correct
- [ ] Structured data (product, article) present
- [ ] Sitemap accessible
- [ ] Robots.txt correct

Performance

- [ ] PageSpeed Insights score > 80
- [ ] No render-blocking resources
- [ ] Images optimized (WebP, correct sizes)
- [ ] JS deferred where possible
- [ ] CSS minified

Apps & Integrations

- [ ] All installed apps still work
- [ ] No app script conflicts
- [ ] App blocks render correctly
- [ ] Analytics tracking fires
- [ ] Email notifications configured

Content

- [ ] All placeholder text replaced
- [ ] Test products/collections removed or hidden
- [ ] Legal pages (privacy, terms) present
- [ ] Contact information accurate
- [ ] Social links correct

Takeaways

- Test functionality, design, technical, SEO, performance, apps, content.
- Use dev store or theme duplicate — never test on live.
- Keep backup for rollback.
- Document customizations for future reference.

What's Next

You've mastered Liquid fundamentals, OS 2.0 architecture, and production customization patterns. To accelerate your Shopify development:

- **GPLMart's Liquid snippet library** at gplmart.com — hundreds of production-ready snippets for product pages, collections, carts, and more. Drop them in, customize the settings, ship faster.
- **Advanced topics to explore**: Shopify Functions (checkout customization), Storefront API (headless), theme app extensions, metafields and metaobjects for structured data, and Liquid performance optimization at scale.

The best way to learn is to build. Pick a customization from Chapter 14, implement it on your dev store, and iterate until it's production-ready. Every section you build makes the next one faster.

Shopify Liquid Mastery — GPLMart original content. Licensed under GPL-3.0-or-later.